each flow ends, the recorded results is the overhead of real-time feature extraction for each packet.

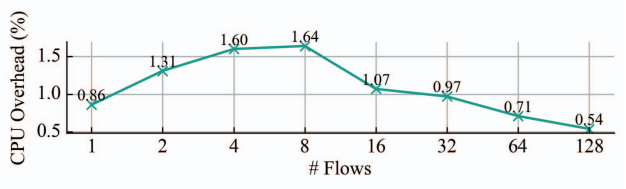


Fig. 9: CPU Overhead of Feature Extraction

As shown in Figure 9, the CPU overhead remains between 0.54% and 1.64%. If the number of concurrent flows does not exceed 8 (total CPU cores), the CPU overhead increases as the number of flows grows because the total bandwidth becomes larger. After exceeding the limit, each flow not only has to compete for bandwidth, but also for CPU cores, so the total bandwidth is declining and the CPU overhead decreases even the number of flows grows.

E. Overlooked Issues in Existing Methods

The worst-case spatial complexity of DT used in [12] is $O(2^n)$, where n represents the depth [43]. Although tree pruning can reduce nodes in the training process, the number of nodes in the trained DT varies even for the same dataset. We train 1024 DT and the training data comprised 80% randomly sampled data from each category of CIC-IDS-2017. As shown in Figure 10, the distribution of the number of nodes is mainly spread between 300 and 600. In order to hot-update

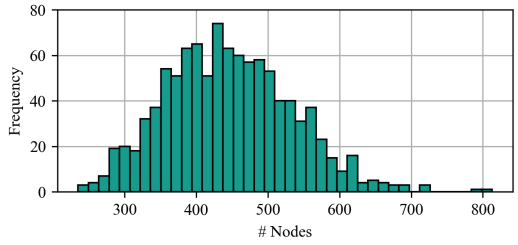


Fig. 10: Distribution of the number of DT nodes.

parameters, eBPF program has to consider the worst case, which introduces exponential memory overheads to the kernel. In addition, the depth of DT also determine its performance. As shown in Figure IX, when using the Top 6 features, if the depth is less than 9, the performance of DT is inferior to our method (NN-int32 Q+Top K). DT implemented in [12] requires at least 4 *int64* arrays. If the depth is set to 10, then the worst-case memory would reach $4 \times 8 \times 2^{10} = 32$ KB. However, our method achieves performance close to that of DT while reducing memory overhead to 5 KB (§ V-D).

Although DT achieves great anomaly detection performance with a simple structure, it still suffer from the problem of concept drift. To illustrate this, we remove one type of intrusion from the training and testing datasets from CIC-IDS-2017 at a time, then train a DT with a depth of 10 and use

TABLE IX: Impact of DT Depth [12]

Depth	Accuracy	Precision	Recall	F1-score
3	0.862	0.599	0.910	0.722
4	0.886	0.990	0.425	0.594
5	0.914	0.829	0.706	0.763
6	0.944	0.799	0.958	0.871
7	0.967	0.898	0.941	0.919
8	0.971	0.966	0.886	0.924
9	**0.983**	**0.964**	**0.951**	**0.957**
10	0.987	0.962	0.972	0.967

the trained DT to detect the removed intrusions. We find that although the average F1-score of the DT on the testing dataset reaches 0.969, the accuracy on each removed intrusion types is low, as shown in Figure 11. We also find that NN generally performs better than DT on the removed intrusion types but still suffer from the problem of concept drift.

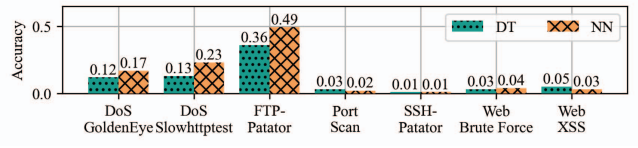


Fig. 11: Accuracy on the removed intrusions

When using non-linear SVM, we find that even with the same training dataset and Top 6 features, the training time of SVM does not stop even after several hours and is much longer compared to DT and NN. DT and NN takes 5s and 71s respectively to achieve the effectiveness shown in Table V. Even we set the maximum number of iterations to 1000 during training, SVM training is still slow. As shown in Table X, results on the testing dataset indicate that non-linear kernels do not improve the performance of the model. Computations involving the exponential function e^x in the *RBF* and *Sigmoid* kernels also pose significant challenges for its integer implementations in eBPF.

TABLE X: Performance of SVM with non-linear kernels

SVM Kernel	Accuracy	Precision	Recall	F1-score	Training Time (s)
Linear	0.262	0.210	0.999	0.348	1289
Polynomial	0.198	0.197	0.999	0.329	1414
RBF	0.222	0.202	0.999	0.336	2147
Sigmoid	0.421	0.196	0.625	0.298	2706

Compared to NN-int8 [14], our method (NN-int32) is not simply increase the length of quantized integers from 8 bits to 32 bits. Hara et al. [14] use *int8* to store the model parameters, while each layer still use *int32* to store the input and output. Thus, for each layer, the input is first quantized into *int8*, then undergoes linear and relu layer, and finally, dequantized to obtain the output in *int32*, which serves as the input for the next layer. The quantization and dequantization steps in each layer add considerable computation overhead and error to the real-time inference. Furthermore, the two steps introduce

extra memory overhead beyond the model parameters [44]. However, our method essentially stores floating-point numbers in the form of fixed-point numbers within int32, eliminating the need for additional quantization and dequantization steps during inference. We design our NN inference algorithm and implementation mechanism to accommodate the limitations of eBPF overlooked in [14]. We significantly improve performance while reducing the computational complexity.

VI. Related Work

Real-time intrusion detection and prevention can protect system from potential impact on its functionality [3]. However, current methods [4]–[7] focus on algorithmic improvements while neglecting the CPU, memory, and context switch overhead during real-time packet capturing. Unlike these methods, we utilize eBPF to offload the intrusion detection model into XDP hook, enabling real-time packet capturing and subsequent analysis within the kernel. We significantly reduces the kernel-user context switch overhead and make it compatible with a wider range of NIC architectures in a cost-effective way.

There are also similar methods that directly use eBPF for intrusion detection. [12] first implements DT in eBPF, but the structure of DT becomes unfixed after each training, presenting challenges in storing and updating it within eBPF. Linear Support Vector Machine employed by [13] may not be suitable for intrusion detection, which is not a straightforward linearly separable problem. NN has fixed structure and strong fitting capabilities. Therefore, NN is widely used in intrusion detection task now [9]. [14] implements NN in eBPF based on *int8* quantization. However, the quantization method introduces significant errors and unnecessarily complicates the implementation process. Moreover, [12]–[14] neglect the critical analyses such as feature importance, overlooked problems like race conditions and limitations in implementing complex algorithms, and the reproduction real-time intrusions for evaluation. We redesign a new NN inference mechanism based on int32 and implement it in eBPF through chained eBPF tail calls. We then propose a thread-safe parameters hot-updating mechanism. Through comprehensive evaluations, we demonstrate that our method achieves performance and inference overhead comparable to the existing methods while reducing memory overhead.

VII. Limitations & Discussions

Since the detection occurs only at the end of a flow, our current implementation is not suitable for intrusions during persistent connections. *Persistent connection* means that it remains open for a long duration without being disconnected. The detection of intrusions in persistent connections requires detecting every packet or flow features within a specific time window. This limitation can be addressed by replacing the training dataset and conducting detection immediately after each packet. Since current mainstream intrusion detection datasets are primarily based on features of the entire flow [33], and conducting detection for each received packet introduces excessive computational overhead, we choose to perform detection only after the completion of a flow.

VIII. Conclusion

We address the overlooked issues in eBPF such as the maximum instructions number, integer-only arithmetic operations, and race conditions. Subsequently, we implement an efficient real-time intrusion detection and defense prototype within the kernel. First, we redesign an NN inference algorithm based on *int32* quantization and integer-only arithmetic. To overcome the maximum eBPF instruction limitation, we decompose the algorithm and employ chained eBPF Tail Calls for real-time inference in XDP. Furthermore, we implement a thread-safe mechanism for hot-updating model parameters. The evaluation results show that our methods reduce memory and inference time overhead while maintains performance comparable to the existing state-of-the-art method using eBPF.

Acknowledgment

We greatly appreciate the insightful feedback from the anonymous reviewers. The research is sponsored by the National Key Research and Development Program of China (2019YFB1804002), the National Natural Science Foundation of China (No.62272495) and the Guangdong Basic and Applied Basic Research Foundation (No.2023B1515020054), and sponsored by Huawei. The corresponding author is Pengfei Chen.

References

[1] Shih-Wei Li, John S Koh, and Jason Nieh. Protecting cloud virtual machines from hypervisor and host operating system exploits. In *28th USENIX Security Symposium (USENIX Security 19)*, pages 1357–1374, 2019.

[2] Amazon. Cloud security software. https://aws.amazon.com/marketplace/solutions/security.

[3] Ravi Sekar and Prem UppuluriR Sekar. Synthesizing fast intrusion prevention/detection systems from high-level specifications. In *8th USENIX Security Symposium (USENIX Security 99)*, 1999.

[4] Yisroel Mirsky, Tomer Doitshman, Yuval Elovici, and Asaf Shabtai. Kitsune: an ensemble of autoencoders for online network intrusion detection. *arXiv preprint arXiv:1802.09089*, 2018.

[5] Steve TK Jan, Qingying Hao, Tianrui Hu, Jiameng Pu, Sonal Oswal, Gang Wang, and Bimal Viswanath. Throwing darts in the dark? detecting bots with limited data using neural data augmentation. In *2020 IEEE symposium on security and privacy (SP)*, pages 1190–1206. IEEE, 2020.

[6] HyungBin Seo and MyungKeun Yoon. Generative intrusion detection and prevention on data stream. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 4319–4335, 2023.

[7] Giorgos Vasiliadis, Spiros Antonatos, Michalis Polychronakis, Evangelos P Markatos, and Sotiris Ioannidis. Gnort: High performance network intrusion detection using graphics processors. In *Recent Advances in Intrusion Detection: 11th International Symposium, RAID 2008, Cambridge, MA, USA, September 15-17, 2008. Proceedings 11*, pages 116–134. Springer, 2008.

[8] iptables. https://linux.die.net/man/8/iptables.

[9] Feng Wei, Hongda Li, Ziming Zhao, and Hongxin Hu. Xnids: Explaining deep learning-based network intrusion detection systems for active intrusion responses. In *32nd USENIX Security Symposium (USENIX Security 23), Anaheim, CA, USA*, 2023.

[10] Matteo Bertrone, Sebastiano Miano, Fulvio Risso, and Massimo Tumolo. Accelerating linux security with ebpf iptables. In *Proceedings of the ACM SIGCOMM 2018 Conference on Posters and Demos*, pages 108–110, 2018.

[11] eBPF. Available at: https://ebpf.io/.

[12] Maximilian Bachl, Joachim Fabini, and Tanja Zseby. A flow-based ids using machine learning in ebpf. *arXiv preprint arXiv:2102.09980*, 2021.

[13] NEMALIKANTI ANAND, MA SAIFULLA, and Pavan Kumar Aakula. High-performance intrusion detection systemusing ebpf with machine learning algorithms. 2023.

[14] Takanori Hara and Masahiro Sasabe. On practicality of kernel packet processing empowered by lightweight neural network and decision tree. In *2023 14th International Conference on Network of the Future (NoF)*, pages 89–97. IEEE, 2023.

[15] Linux. Bpf design q&a. Available at: https://www.kernel.org/doc/html/v5.2/bpf/bpf_design_QA.html.

[16] Jonathan Corbet. Concurrency management in bpf. https://lwn.net/Articles/779120/.

[17] Ahmad Javaid, Quamar Niyaz, Weiqing Sun, and Mansoor Alam. A deep learning approach for network intrusion detection system. In *Proceedings of the 9th EAI International Conference on Bio-inspired Information and Communications Technologies (formerly BIONETICS)*, pages 21–26, 2016.

[18] Jihyun Kim, Jaehyun Kim, Huong Le Thi Thu, and Howon Kim. Long short term memory recurrent neural network classifier for intrusion detection. In *2016 international conference on platform technology and service (PlatCon)*, pages 1–5. IEEE, 2016.

[19] Guansong Pang, Chunhua Shen, Longbing Cao, and Anton Van Den Hengel. Deep learning for anomaly detection: A review. *ACM computing surveys (CSUR)*, 54(2):1–38, 2021.

[20] Cillium. Bpf and xdp reference guide. Available at: https://docs.cilium.io/en/latest/bpf/.

[21] Limin Yang, Wenbo Guo, Qingying Hao, Arridhana Ciptadi, Ali Ahmadzadeh, Xinyu Xing, and Gang Wang. Cade: Detecting and explaining concept drift samples for security applications. In *30th USENIX Security Symposium (USENIX Security 21)*, pages 2327–2344, 2021.

[22] Tcpdump. https://www.tcpdump.org/manpages/tcpdump.1.html.

[23] Mohammad Saiful Islam Mamun Arash Habibi Lashkari, Gerard Draper-Gil and Ali A. Ghorbani. cicflowmeter. Available at: https://www.unb.ca/cic/research/applications.html.

[24] Dongzi Jin, Yiqin Lu, Jiancheng Qin, Zhe Cheng, and Zhongshu Mao. Swiftids: Real-time intrusion detection system based on lightgbm and parallel intrusion detection mechanism. *Computers & Security*, 97:101984, 2020.

[25] Libpcap. https://www.tcpdump.org/manpages/pcap-filter.7.html.

[26] Programming with pcap. Available at: https://www.tcpdump.org/pcap.html.

[27] pidstat. Available at: https://man7.org/linux/man-pages/man1/pidstat.1.html.

[28] Yang Zhou, Zezhou Wang, Sowmya Dharanipragada, and Minlan Yu. Electrode: Accelerating distributed protocols with ebpf. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, pages 1391–1407, 2023.

[29] Toke Høiland-Jørgensen, Jesper Dangaard Brouer, Daniel Borkmann, John Fastabend, Tom Herbert, David Ahern, and David Miller. The express data path: Fast programmable packet processing in the operating system kernel. In *Proceedings of the 14th international conference on emerging networking experiments and technologies*, pages 54–66, 2018.

[30] Tc-bpf. Available at: https://man7.org/linux/man-pages/man8/tc-bpf.8.html.

[31] Sebastiano Miano, Matteo Bertrone, Fulvio Risso, Massimo Tumolo, and Mauricio Vásquez Bernal. Creating complex network services with ebpf: Experience and lessons learned. In *2018 IEEE 19th International Conference on High Performance Switching and Routing (HPSR)*, pages 1–8. IEEE, 2018.

[32] John H Ring IV, Colin M Van Oort, Samson Durst, Vanessa White, Joseph P Near, and Christian Skalka. Methods for host-based intrusion detection with deep learning. *Digital Threats: Research and Practice (DTRAP)*, 2(4):1–29, 2021.

[33] Iman Sharafaldin, Arash Habibi Lashkari, and Ali A Ghorbani. Toward generating a new intrusion detection dataset and intrusion traffic characterization. *ICISSp*, 1:108–116, 2018.

[34] httperf. Available at: https://github.com/httperf/httperf.

[35] iperf. Available at: https://iperf.fr.

[36] GoldenEye. Available at: https://github.com/jseidl/GoldenEye.

[37] Slowhttptest. Available at: https://github.com/shekyan/slowhttptest.

[38] patator. Available at: https://github.com/lanjelot/patator.

[39] DVWA. Damn vulnerable web app. Available at: https://github.com/digininja/DVWA.

[40] selenium. Available at: https://selenium-python.readthedocs.io/.

[41] Brian W Matthews. Comparison of the predicted and observed secondary structure of t4 phage lysozyme. *Biochimica et Biophysica Acta (BBA)-Protein Structure*, 405(2):442–451, 1975.

[42] perf. Available at: https://perf.wiki.kernel.org/index.php/Tutorial.

[43] Thomas H Cormen, Charles E Leiserson, Ronald L Rivest, and Clifford Stein. *Introduction to algorithms*. MIT press, 2022.

[44] Hao Wu, Patrick Judd, Xiaojie Zhang, Mikhail Isaev, and Paulius Micikevicius. Integer quantization for deep learning inference: Principles and empirical evaluation, 2020.